

LAB GUIDELINES

Lab 15 — Virtualization with KVM / QEMU

Maps to CompTIA Server+ objective **2.5** — Summarize the purpose and operation of virtualization (host vs. guest; virtual networking — bridged, NAT, vNICs, virtual switches; resource allocation — CPU, memory, disk, NIC, overprovisioning, scalability; management interfaces for VMs; cloud models — public, private, hybrid).

> **Killercoda caveat:** the playground is itself a VM. Nested KVM may or may not be available. Where hardware virt is disabled, QEMU falls back to TCG (slower but functional) — the commands and concepts are the same.

Run all commands on <https://killercoda.com/playgrounds/scenario/ubuntu>

Required software (free, apt): qemu-system-x86, qemu-utils, libvirt-daemon-system, virtinst, bridge-utils

```
apt update && apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system virtinst bridge-utils
systemctl start libvirtd
virsh version
```

Step 1 — Host vs. guest

```
egrep -c '(vmx|svm)' /proc/cpuinfo # >0 = host CPU supports hardware virt
kvm-ok 2>/dev/null || echo "Hardware virt may be unavailable – TCG fallback will be used"
```

The **host** runs the hypervisor and owns hardware. **Guests** are VMs scheduled onto the host.

Step 2 — Create a virtual disk

```
mkdir -p /var/lib/libvirt/images
qemu-img create -f qcow2 /var/lib/libvirt/images/vm1.qcow2 5G
qemu-img info /var/lib/libvirt/images/vm1.qcow2
```

qcow2 is **thin-provisioned** — the file grows as the guest writes. This enables **overprovisioning** of disk.

Step 3 — Define a VM (without booting on this VM-in-a-VM env)

```
virt-install \
  --name vm1 --memory 512 --vcpus 1 \
  --disk path=/var/lib/libvirt/images/vm1.qcow2,format=qcow2 \
  --network network=default \
  --osinfo detect=on,name=generic \
  --import --noautoconsole --dry-run 2>&1 | tail
virsh list --all
```

Map every flag to objective 2.5: --memory = memory allocation, --vcpus = CPU allocation, --disk = disk allocation, --network network=default = NAT vNIC on the virtual switch virbr0.

Step 4 — Inspect virtual networking

```
virsh net-list --all
virsh net-dumpxml default
brctl show virbr0 2>/dev/null || bridge link
ip addr show virbr0
```

Three networking modes the exam tests:

Mode	What the guest sees	Where you'd use it
Bridged (direct)	IP from the same LAN as the host	Production VMs on the office network
NAT	Private IP, host NATs to outside	Lab VMs, desktops
Isolated	Only other VMs on the same vSwitch	Inter-VM lab networks

Step 5 — Add a second vNIC (multiple virtual switches)

```
virsh net-define <(cat <<'EOF'
<network>
  <name>internal</name>
  <bridge name='virbr-int' stp='on' delay='0' />
  <ip address='192.168.222.1' netmask='255.255.255.0' />
</network>
EOF
)
virsh net-start internal
virsh net-list
```

`virbr-int` is a second **virtual switch**. Attach a vNIC from any VM to it and that VM gets a leg on the 192.168.222.0/24 network.

Step 6 — Resource allocation, overprovisioning, scalability

```
nproc          # physical CPUs available
free -h        # physical RAM available
df -h /var/lib/libvirt/images
```

Rules of thumb:

- **CPU**: 4:1 vCPU:pCPU ratio is fine for general workloads, 1:1 for latency-sensitive DBs.
- **Memory**: never overprovision past physical RAM minus a buffer for the host.
- **Disk**: thin-provision freely but **monitor utilisation** (Lab 12) — running the host filesystem out of space crashes every guest.
- **NIC**: aggregate guest bandwidth must not exceed host NIC throughput.

Step 7 — Management interfaces for VMs

Interface	Tool
CLI	<code>\virsh`, \virt-install`</code>
Local GUI	<code>\virt-manager`</code>
Web	Cockpit, Proxmox web UI, oVirt, vSphere web
API	libvirt API, VMware vSphere API, AWS / Azure

Step 8 — Snapshots (VM-level point-in-time)

```
virsh snapshot-create-as vm1 snap1 "pre-patch" --disk-only --atomic 2>/dev/null || \
echo "(no running VM – on a running guest this would create snap1.qcow2 backing chain)"
virsh snapshot-list vm1 2>/dev/null
```

Snapshots are the fastest rollback for patches (Lab 24) but are **not a backup** — the backing file lives on the same host.

Step 9 — Cloud delivery models

Model	Definition	Example
Public	Provider-owned, multi-tenant, internet-reachable	AWS EC2, Azure VMs, GCP Compute Engine
Private	Single-tenant, on-prem or hosted, full control	On-prem VMware, Proxmox, OpenStack
Hybrid	Mix of public + private, workloads span both	AWS Outposts, Azure Arc, Anthos

Lab environments

Technology	Platform	Where
Linux VM / KVM	Killercoda Ubuntu playground	https://killercoda.com/playgrounds/scenario/ubuntu
Containers (Docker)	Docker Desktop	https://www.docker.com/products/docker-desktop/
Kubernetes	Killercoda Kubernetes playground	https://killercoda.com/playgrounds/scenario/kubernetes

Container & Kubernetes assets (bonus)

A VM virtualises hardware and runs a full guest OS; a container is OS-level virtualization that shares the host kernel. Compare both with the files in `[assets/](assets/)`:

- `[`assets/Dockerfile`](assets/Dockerfile)` + `[`assets/app.py`](assets/app.py)` — build a small app image with **Docker Desktop**:

```
docker build -t serverplus-app .
docker run -d -p 8000:8000 serverplus-app
```

- `[`assets/deployment.yaml`](assets/deployment.yaml)` + `[`assets/service.yaml`](assets/service.yaml)` — run and scale it on the **Killercoda Kubernetes** playground:

```
kubectl apply -f deployment.yaml -f service.yaml
kubectl scale deployment serverplus-app --replicas=4
kubectl get pods,svc -o wide
```

Free online tools

- **libvirt docs** — <https://libvirt.org/>
- **QEMU manual** — <https://www.qemu.org/docs/master/>
- **Cockpit (web management)** — <https://cockpit-project.org/>
- **Proxmox VE (free hypervisor)** — <https://www.proxmox.com/>
- **Docker Desktop** — <https://www.docker.com/products/docker-desktop/>
- **Killercoda Kubernetes playground** — <https://killercoda.com/playgrounds/scenario/kubernetes>

What you learned

- Host vs. guest, hypervisor types, and the role of qcow2.
- Building a VM definition with ``virt-install`` and mapping every flag to objective 2.5.
- Bridged vs. NAT vs. isolated virtual switches, and how to add a second one.
- CPU/memory/disk/NIC overprovisioning rules of thumb.
- Management interface options and where snapshots fit (and don't) in backup.
- Public / private / hybrid cloud model definitions.